

Circular Queue Concept

A comprehensive technical reference on one of the most fundamental data structures in computer science — covering theory, implementation, real-world applications, and performance optimization.

[DATA STRUCTURES & ALGORITHMS](#)

[BEGINNER TO INTERMEDIATE](#)

[DSA – QUEUE VARIANTS](#)

Developed by Talenciaglobal

Topic Classification

Course Metadata

Category	Data Structures / Linear Data Structures
Domain	DSA – Queue Variants
Topic Type	ADT Implementation Pattern
Difficulty	Beginner to Intermediate

Recommended Learning Path

- 01

Linear Queue Limitations
- 02

Circular Queue Concept
- 03

Array Implementation
- 04

Full / Empty Handling
- 05

Use Cases & Advanced Variants

 Prerequisites: Basic queue (FIFO), arrays, and the modulo operator. Industry Relevance: **High** — used in ring buffers, network packet queues, audio/video streaming, and logging systems.

What Is a Circular Queue?

A circular queue is a queue whose last position is connected back to the first position to form a circle. It is implemented using a fixed-size array and two pointers — **front** and **rear** — that wrap around using modulo arithmetic.

This structure reuses empty slots created by dequeue operations, overcoming the fundamental limitation of a simple linear array queue. The result is a highly efficient, predictable, and real-time-safe data structure.

Simple Explanation: The Circular Seat Analogy

Imagine a row of **5 seats** numbered 0 to 4. People arrive and sit from left to right. When someone leaves from the front, that seat becomes empty.

Instead of moving everyone forward — which is slow and costly — the next person can sit at the "new" front if empty seats exist at the beginning. **The queue wraps around, like a circle of seats.**

- ✓ This simple insight eliminates the $O(n)$ shifting cost of a linear queue and enables $O(1)$ operations for all queue actions.

Seat 0

Reusable after dequeue

Seat 1–3

Active elements

Seat 4

Wraps to Seat 0

Core Purpose & Key Terminology



Efficient Space Usage

Reuses vacated slots without shifting elements, maximizing utilization of a fixed-size array.



Fixed Memory Footprint

Predictable, no dynamic allocation — critical for embedded and real-time systems.



O(1) Operations

Both enqueue and dequeue execute in constant time, regardless of queue size.



Ideal for Buffers

Perfectly suited for streaming data, producer-consumer pipelines, and real-time logging.

Term	Definition
Front	Index of the first element (oldest) in the queue.
Rear	Index of the next free slot (or last element, depending on convention).
Modulo (%)	Operation that wraps indices: $(i + 1) \% \text{capacity}$.
Circular Buffer	Alternate name, often used in embedded systems and data streaming contexts.
Wasted Slot	One slot left unused to distinguish full from empty (common convention).
Count	Alternative method to track number of elements, avoiding the wasted slot.

Why Does It Exist? Problems It Solves

Problem	Linear Array Queue	Circular Queue
Front moves forward after dequeues; rear reaches end	Empty space at front cannot be reused — "queue full" even though slots are free	Enqueue wraps around, reusing empty front slots
Shifting elements after dequeue	$O(n)$ per dequeue if shifting — highly inefficient	No shifting required; $O(1)$ always
Fixed memory waste	Wasted space at front accumulates over time	Maximum capacity fixed but fully utilised

Technical Driver

Embedded systems need predictable memory and $O(1)$ operations without dynamic allocation.

Business Driver

Streaming media (audio/video) requires constant-latency buffering — circular buffer is the industry standard.

Industry Driver

Network routers, audio DSP, real-time logging, and producer-consumer threads all rely on this pattern.

Real-World Motivation: Industry Statistics

44.1K

Samples/Second

Standard audio sampling rate — circular buffers prevent overflow and underflow in every consumer audio device.

~1M

Packets/Second

Modern network routers process up to 1 million packets per second using fixed-size circular queues with tail-drop policies.

6M+

Lines of Code

The Linux kernel — which uses `kfifo` circular buffers extensively — contains over 6 million lines relying on this pattern.

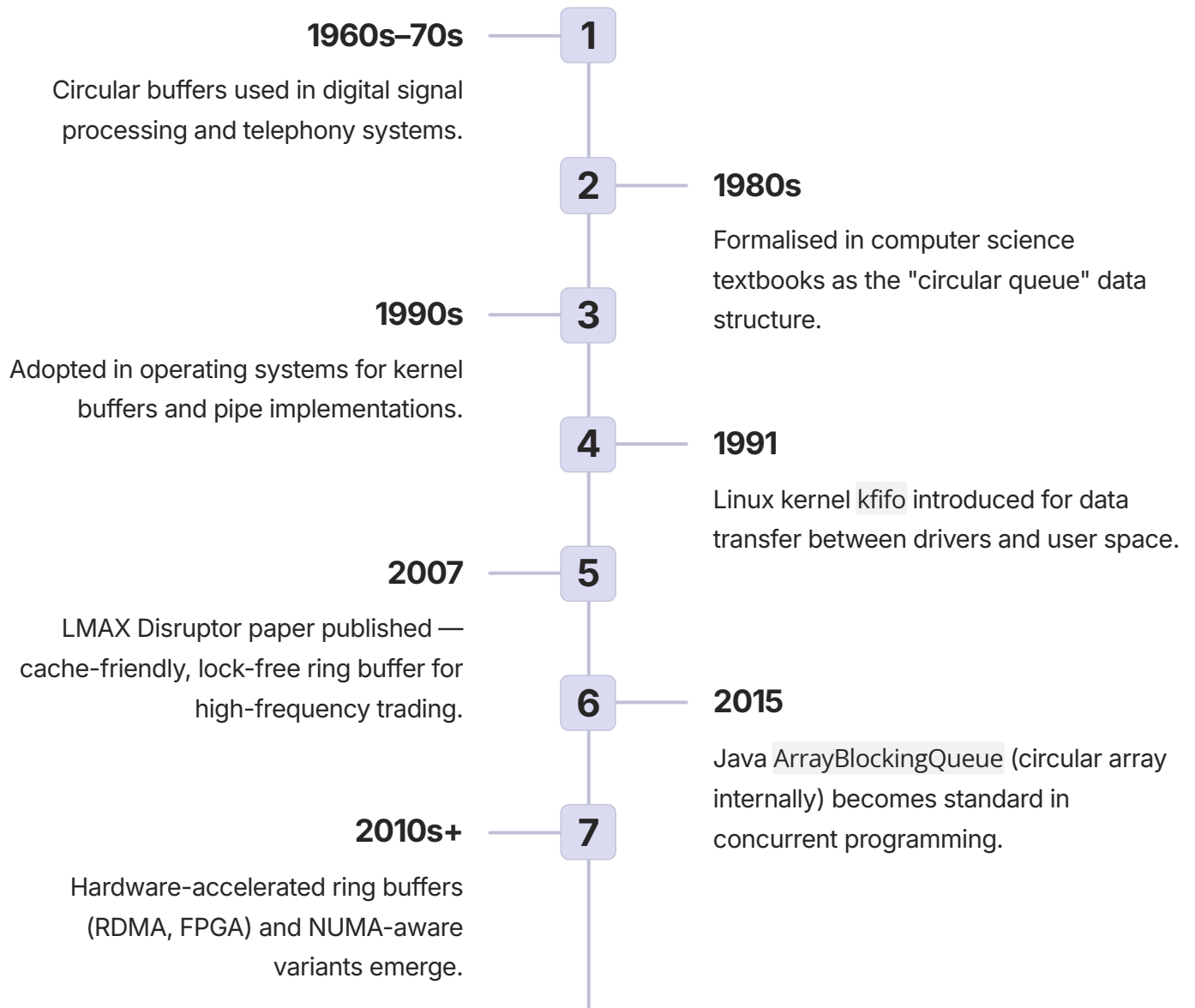
6M

Msgs/Second

LMAX Disruptor, a lock-free ring buffer, achieves 6 million messages per second in financial trading systems.

i Without a circular queue, engineers would either shift elements after each dequeue ($O(n)$ — infeasible for real-time), use a linked queue (dynamic allocation, cache misses), or accept that a linear array queue becomes unusable once `rear` reaches the end.

Evolution & History



Today, the circular queue is a standard pattern in every programmer's toolkit, implemented in standard libraries across Java, Python, C++, and virtually every systems programming language.

Future Trends

NUMA-Aware Ring Buffers

Optimised for multi-socket systems where memory locality across CPU nodes is critical for performance.

Persistent Circular Buffers

Used as flight recorders in automotive and aerospace — logging to non-volatile memory for crash analysis.

Wait-Free Queues for AI/ML

Real-time AI/ML inference pipelines require wait-free circular queues to guarantee bounded latency.

Core Mechanism: How It Works

A circular queue uses a **fixed-size array of capacity N** with two indices: **front** (points to the first element) and **rear** (points to the next free slot). Initially, **front = rear = 0**.

Enqueue Operation

```
array[rear] = value  
rear = (rear + 1) % N
```

Dequeue Operation

```
value = array[front]  
front = (front + 1) % N
```

Step-by-Step Trace (N=5)

Initial: [_ _ _ _ _] f=0, r=0

+A: [A, _ _ _ _] f=0, r=1

+B: [A, B, _ _ _] f=0, r=2

+C: [A, B, C, _ _] f=0, r=3

-A: [A, B, C, _ _] f=1, r=3

+D: [A, B, C, D, _] f=1, r=4

+E: [A, B, C, D, E] f=1, r=0

FULL: rear wraps to 0!

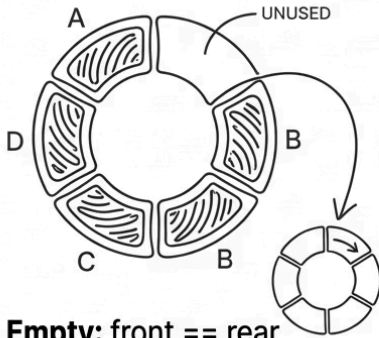
- ❏ The modulo operator **%** is the mathematical engine of the circular queue. It ensures indices never exceed the array bounds and wrap seamlessly from the last slot back to the first.

Full / Empty Detection Conventions

CONVENTION 1 (WASTE ONE SLOT)

Empty: $\text{front} == \text{rear}$

Full: $(\text{rear} + 1) \% N == \text{front}$



Empty: $\text{front} == \text{rear}$

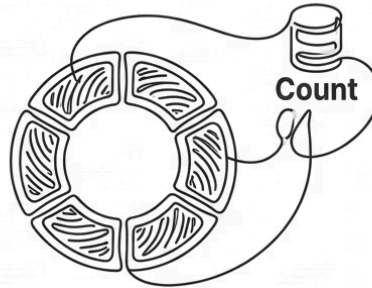
Full: $(\text{rear} + 1) \% N == \text{front}$

One slot always unused,
max elements = $N-1$

CONVENTION 2 (MAINTAIN COUNT)

Empty: $\text{count} == 0$

Full: $\text{count} == N$



Empty: $\text{count} == 0$

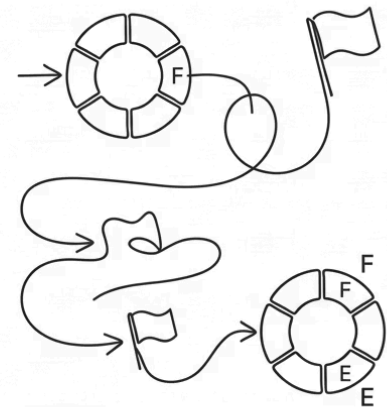
Full: $\text{count} == N$

Requires updating count on
every operation, uses all N slots

CONVENTION 3 (USE A FLAG)

Boolean 'full' set on last
enqueue, cleared on dequeue

Rare and error-prone



Rare and error-prone



Recommended: Convention 1 (Waste One Slot) — Simplest to implement and reason about.

Empty: $\text{front} == \text{rear}$. Full: $(\text{rear} + 1) \% N == \text{front}$. One slot is always unused; maximum elements = $N-1$.

Operation Steps & Complexity

Enqueue (offer) — Convention 1

1 Check Full

If $(\text{rear} + 1) \% N == \text{front}$ → queue is full; return false.

2 Write Value

$\text{arr}[\text{rear}] = \text{value}$

3 Advance Rear

$\text{rear} = (\text{rear} + 1) \% N$

Dequeue (poll) — Convention 1

1 Check Empty

If $\text{front} == \text{rear}$ → queue is empty; return null.

2 Read Value

$\text{value} = \text{arr}[\text{front}]$

3 Advance Front

$\text{front} = (\text{front} + 1) \% N$

4 Return

Return the retrieved value.

Complexity Summary

Enqueue

$O(1)$ time

Dequeue

$O(1)$ time

Peek

$O(1)$ time

Size

$(\text{rear} - \text{front} + N) \% N$

Space

$O(N)$ array

Types & Variants

Fixed-Size (Waste Slot)

Standard implementation with simple full/empty detection. Most common in embedded and general-purpose buffers.

Circular Queue with Count

No wasted slot; uses a separate counter. Use when maximum capacity must be fully utilised.

Lock-Free Circular Queue

Uses atomic operations — no mutexes. Ideal for multi-producer/multi-consumer scenarios (e.g., LMAX Disruptor).

Overwriting Circular Queue

When full, newest data overwrites oldest. Used in real-time data streams, live audio, and sensor logs.

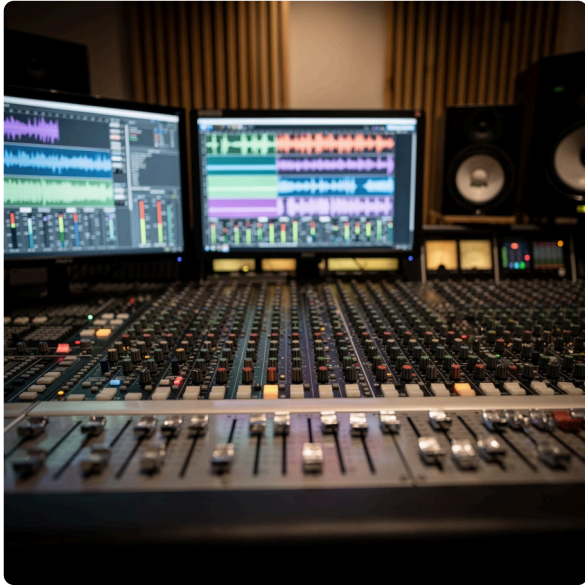
Blocking Circular Queue

Waits when empty or full using condition variables. Standard for producer-consumer thread patterns.

Memory-Mapped Circular Queue

Stored in shared memory for inter-process communication. Enables fast IPC without kernel involvement.

Real-World Use Case 1: Audio Playback Buffer



EMBEDDED AUDIO / CONSUMER ELECTRONICS

Business Problem

A microphone produces **48,000 samples per second**; the audio codec consumes at the same rate. The buffer must never overflow or underflow — any gap produces audible artifacts.

Solution

A circular queue (ring buffer) of **4,096 samples**; the producer writes, the consumer reads. If full, the producer blocks (overwrite disallowed in this configuration).

```
#define BUFFER_SIZE 4096
int16_t buffer[BUFFER_SIZE];
int front = 0, rear = 0;

bool enqueue(int16_t sample) {
    int next = (rear + 1) % BUFFER_SIZE;
    if (next == front) return false;
    buffer[rear] = sample;
    rear = next;
    return true;
}
```

✓ Benefits: $O(1)$, no dynamic memory, real-time safe. Lesson: Use double buffering or DMA for even lower latency.

Real-World Use Case 2: Network Packet Queue

NETWORKING — ROUTER / SWITCH

Business Problem

Incoming packets arrive faster than they can be processed. A fixed-size queue is required; when full, new packets are dropped (tail-drop policy). Industry data: a single 100 Gbps router port may receive **over 148 million packets per second**.

Solution

```
public class PacketQueue {  
    private static final int MAX = 1000;  
    private Packet[] buffer = new Packet[MAX];  
    private int front = 0, rear = 0;  
  
    public boolean offer(Packet p) {  
        int next = (rear + 1) % MAX;  
        if (next == front) return false; // drop  
        buffer[rear] = p;  
        rear = next;  
        return true;  
    }  
}
```

⚠ Lesson: Tail-drop can cause global TCP synchronisation (incast). Use RED (Random Early Detection) for production deployments.



Real-World Use Case 3: Real-Time Logging (Overwriting Buffer)

EMBEDDED / AUTOMOTIVE BLACK BOX

Business Problem

Keep the **last 10,000 log messages**; when full, overwrite the oldest. Used for post-crash analysis in automotive ECUs and avionics flight recorders.

Solution: Overwriting Circular Buffer

```
template<typename T, int N>
class OverwriteCircularBuffer {
    T data[N];
    int head = 0, tail = 0;
    bool full = false;
public:
    void push(const T& val) {
        data[head] = val;
        if (full) tail = (tail+1) % N;
        head = (head + 1) % N;
        full = head == tail;
    }
};
```

Key Benefits

→ Always Latest N Entries

Guarantees the most recent data is always available for analysis.

→ Zero Allocation

No heap allocation at runtime — critical for safety-certified embedded systems.

→ O(1) Always

Constant-time push regardless of buffer state.

- 📄 For debugging, also store timestamps with each entry. Exercise caution with concurrent access — use atomic operations or a mutex.

Implementations: Java — Circular Queue (Waste Slot)

A generic, production-quality circular queue implementation in Java using the waste-slot convention for full/empty detection.

```
public class CircularQueue<T> {
    private Object[] arr;
    private int front, rear;
    private int capacity;

    public CircularQueue(int capacity) {
        this.capacity = capacity;
        arr = new Object[capacity];
        front = rear = 0;
    }

    public boolean enqueue(T item) {
        int next = (rear + 1) % capacity;
        if (next == front) return false; // full
        arr[rear] = item;
        rear = next;
        return true;
    }

    @SuppressWarnings("unchecked")
    public T dequeue() {
        if (front == rear) return null; // empty
        T item = (T) arr[front];
        arr[front] = null; // help GC
        front = (front + 1) % capacity;
        return item;
    }

    public T peek() {
        if (front == rear) return null;
        return (T) arr[front];
    }

    public boolean isEmpty() { return front == rear; }
    public boolean isFull() { return (rear + 1) % capacity == front; }
    public int size() { return (rear - front + capacity) % capacity; }
}
```

- ✔ Setting `arr[front] = null` after dequeue is a Java best practice — it releases the object reference and allows the garbage collector to reclaim memory.

Implementations: Python & C++ Variants

Python — Circular Queue with Count (No Wasted Slot)

```
class CircularQueueCount:
    def __init__(self, k: int):
        self.data = [None] * k
        self.capacity = k
        self.front = 0
        self.rear = 0
        self.size = 0

    def enqueue(self, value: int) -> bool:
        if self.size == self.capacity:
            return False
        self.data[self.rear] = value
        self.rear = (self.rear + 1) % self.capacity
        self.size += 1
        return True

    def dequeue(self) -> int:
        if self.size == 0:
            return -1
        val = self.data[self.front]
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
        return val
```

C++ — Lock-Free Circular Queue (Atomics)

```
template<typename T, int N>
class LockFreeCircularQueue {
    T buffer[N];
    std::atomic<int> head{0}, tail{0};
public:
    bool enqueue(const T& val) {
        int t = tail.load(
            std::memory_order_relaxed);
        int next = (t + 1) % N;
        if (next == head.load(
            std::memory_order_acquire))
            return false; // full
        buffer[t] = val;
        tail.store(next,
            std::memory_order_release);
        return true;
    }
    bool dequeue(T& out) {
        int h = head.load(
            std::memory_order_relaxed);
        if (h == tail.load(
            std::memory_order_acquire))
            return false; // empty
        out = buffer[h];
        head.store((h + 1) % N,
            std::memory_order_release);
        return true;
    }
};
```

Hands-On Labs

1

Lab 1: Basic Circular Queue (Beginner)

Goal: Implement `CircularQueue` with `enqueue`, `dequeue`, `isFull`, `isEmpty`, `size`.

Test Sequence (capacity=3):

- Enqueue 10, 20, 30 → queue full
- Enqueue 40 → False (full)
- Dequeue → 10
- Enqueue 40 → True (wraps around)
- Dequeue all → 20, 30, 40

Learning Outcome: Understand modulo wrap and full/empty logic.

2

Lab 2: Producer-Consumer Buffer (Intermediate)

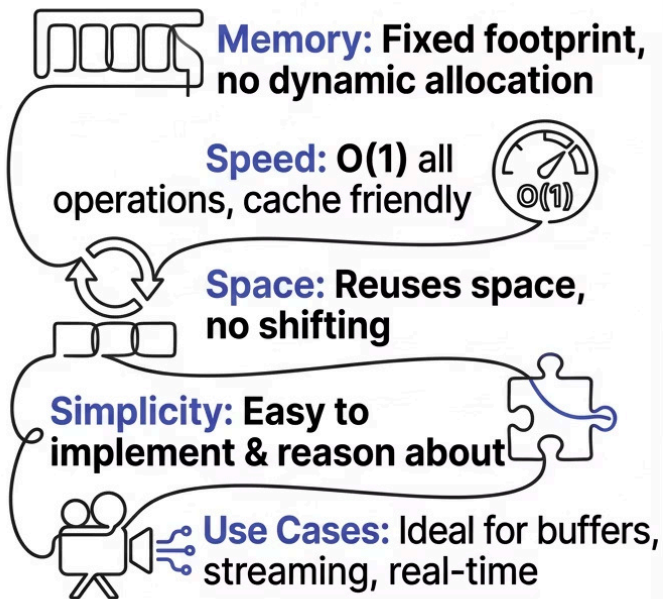
Goal: Simulate an audio buffer with two threads.

- Create circular queue of size 10
- Producer thread enqueues numbers 1..100
- Consumer thread dequeues and prints
- Use mutex + condition variable for blocking behaviour

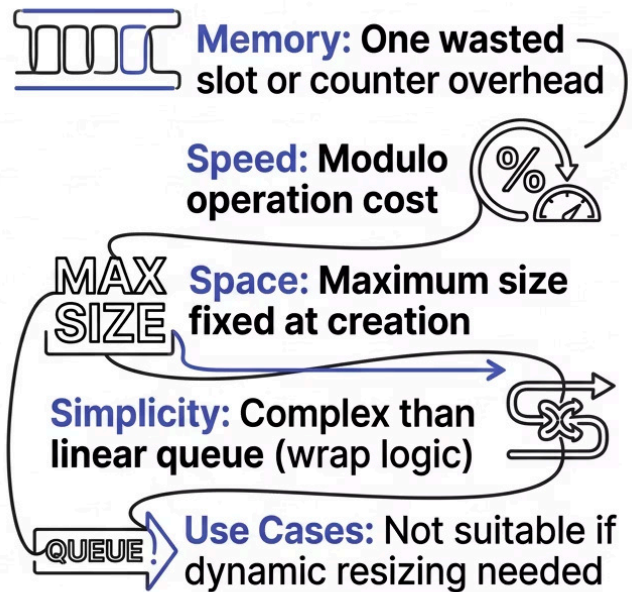
Learning Outcome: Circular queue as a synchronised, thread-safe buffer in a real producer-consumer scenario.

Advantages & Disadvantages

ADVANTAGES



DISADVANTAGES



- i** The circular queue's fixed-capacity constraint is simultaneously its greatest strength (predictability, real-time safety) and its primary limitation (no elastic growth). Choose accordingly based on your system's requirements.

Performance Considerations

Key Performance Metrics (C++ on Modern CPU)

Operation	Latency
Enqueue / Dequeue (warm cache)	~3–5 ns
Full / Empty check	~1 ns
Size computation (with modulo)	~5 ns

Bottlenecks

- **Non-Power-of-Two Capacity**
Integer division is slower than bitwise AND. Use power-of-two sizes whenever possible.
- **Cache False Sharing**
When `front` and `rear` share a cache line in concurrent access, performance degrades significantly.
- **Multi-Thread Contention**
Without proper memory ordering, concurrent access causes data corruption and undefined behaviour.

Optimisation Techniques

Power-of-Two Mask

```
mask = capacity - 1  
rear = (rear+1) &  
mask
```

Replaces division with bitwise AND.

Cache Line Padding

Pad `front` and `rear` to 64 bytes apart to eliminate false sharing in concurrent queues.

Prefetching

Use `__builtin_prefetch` to prefetch the next element before it is needed.

Batch Operations

Process multiple elements per lock acquisition to increase throughput in high-contention scenarios.

Scalability & Reliability

Scaling Strategies

Strategy	Circular Queue Behaviour
Vertical	Increase capacity — requires reallocation and ordered copy. $O(n)$ complexity.
Horizontal	Partition data across multiple queues (sharding). Effective for parallel pipelines.
Elastic	Not naturally elastic. Use a linked queue if unbounded growth is required.

Growing a Circular Queue

If dynamic growth is required, allocate a larger array and copy elements in correct order starting from `front`, wrapping around as needed. Reset `front = 0`, `rear = size`. Complexity: $O(n)$.

Reliability Policies

Overflow Handling

Define a clear policy: drop new data, block producer, or overwrite oldest. Document the choice explicitly.

Underflow Handling

Return an error code, throw an exception, or block the consumer — depending on the application's real-time requirements.

Reset

To clear the queue, simply set `front = rear = 0`. $O(1)$ operation.

Design & Architectural Considerations



Capacity Choice

Prefer **power-of-two** capacity for faster modulo via bitwise AND. Arbitrary sizes are acceptable but incur a small division cost.



Full/Empty Method

Waste-slot is simpler and preferred. Use a count variable only when every slot must be utilised (e.g., exact-capacity requirements).



Overwrite Policy

Overwrite oldest for real-time logs; drop for networking; block for producer-consumer. The policy must be explicit and documented.



Concurrency Model

Use lock-free (atomics) for high throughput; lock-based (mutex) for simplicity; single-threaded for embedded systems.

Circular Queue vs. Linked Queue — Trade-off Analysis

Factor	Circular Queue (Array)	Linked Queue
Memory Overhead	None (except array)	Per-node pointer (8–16 bytes)
Cache Locality	Excellent	Poor (pointer chasing)
Max Size	Fixed at creation	Unlimited
Allocation	None (pre-allocated)	Per enqueue (malloc)
Real-Time Friendly	Yes	No (malloc may block)

Security Considerations

Buffer Overflow (Index Out of Bounds)

If modulo is not applied correctly, indices may access memory beyond the array boundary. **Always use modulo on every index update.**

Race Condition

Concurrent enqueue/dequeue without locks leads to data corruption and undefined behaviour. Use `volatile`, `atomics`, or `mutexes` for shared queues.

Denial of Service

Unbounded blocking on a full/empty queue can cause thread starvation. Mitigate with timeouts on all blocking operations.

Information Leak

Old data not cleared in an overwriting queue may be read by subsequent consumers. Apply zeroisation (`memset` or explicit overwrite) when handling sensitive data.

- ⊗ Hardening checklist: Always validate `capacity > 0`. Use atomic operations for shared memory. Clear memory on dequeue if the queue holds sensitive or classified data.

Best Practices



DO

→ Power-of-Two Capacity

Use bitwise AND (`& mask`) instead of modulo for maximum performance on hot paths.

→ Encapsulate Indices

Keep `front` and `rear` private; expose only ADT methods (`offer`, `poll`, `peek`).

→ Static Allocation for Embedded

Prefer static allocation and compile-time capacity for safety-critical embedded systems.

→ Memory Barriers for Concurrency

Use `std::memory_order_release/acquire` or equivalent in multi-threaded environments.



DON'T

→ Rely on Integer Overflow

Never rely on integer overflow to wrap indices. Always use explicit modulo or bitwise AND.

→ Use for Unbounded Data

Do not use a circular queue for variable-sized data with no maximum bound — use a linked queue instead.

→ Forget Edge Cases

Always handle both full and empty conditions explicitly. Missing either leads to silent data corruption.

→ Ambiguous Full/Empty

Do not use `rear == front` for both empty and full without a wasted slot or counter — this is undefined behaviour.

Common Mistakes & Pitfalls

Beginner

Mistake	Fix
Off-by-one in full condition	Use <code>(rear+1) % N == front</code>
Not resetting front/rear after clearing	Reset both to 0
Using % with negative numbers	Ensure indices are always non-negative

Intermediate

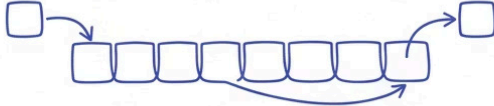
Mistake	Fix
Size calculation wrong	<code>(rear - front + N) % N</code>
Overflow of rear without modulo	Always apply modulo on every update
Overwriting queue loses data unexpectedly	Document and enforce overwrite policy

Advanced

Mistake	Fix
False sharing in concurrent ring buffer	Pad front and rear to different cache lines (64 bytes)
ABA problem in lock-free queue	Use double-width CAS or hazard pointers
Memory reordering	Use <code>std::memory_order_release/acquire</code>

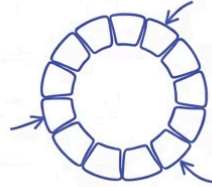
Comparison with Alternative Data Structures

Linear Array Queue



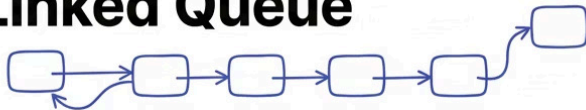
Simpler logic, wastes space after dequeues needs shifting, use for teaching or very small N

Circular Queue



Reuses space $O(1)$, fixed capacity modulo overhead, use for buffers and real-time

Linked Queue

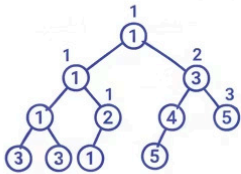


Unlimited size $O(1)$, per-node allocation cache misses, use when max size unknown

Deque as Queue

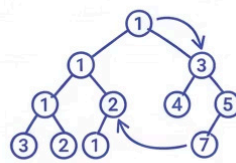


Double-ended can be array of blocks, more complex, use when need both ends



Priority Queue

Ordered by priority, $O(\log n)$ operations, use of scheduling not FIFO



Priority Queue

Ordered by priority, $O(\log n)$ operations, use for sched not FIFO

- The circular queue occupies the optimal middle ground between the simplicity of a linear array queue and the flexibility of a linked queue — making it the default choice for any fixed-capacity, performance-sensitive application.

Learning Summary: Key Takeaways

1 Circular Queue Reuses Empty Slots

By wrapping indices, vacated slots are reclaimed without shifting — the core innovation over linear queues.

2 Modulo Arithmetic Enables the Wrap

$(i + 1) \% N$ is the single mathematical operation that makes the circular structure possible.

3 Empty: `front == rear`; Full: `(rear+1)%N == front`

These two conditions (waste-slot convention) are the foundation of all circular queue logic.

4 All Operations Are $O(1)$

Enqueue, dequeue, peek, and size all execute in constant time — no loops, no shifting.

5 Fixed Capacity — Decide at Creation

The size is immutable after creation. Choose carefully based on worst-case throughput requirements.

6 Power-of-Two Enables Bitwise Optimisation

Replace $\% N$ with $\& \text{mask}$ for a measurable performance gain on hot paths.

7 Overwriting Variant Keeps Latest N Elements

Critical for real-time logging, flight recorders, and live sensor data where recency matters more than completeness.

Quick Reference Cheat Sheet

CIRCULAR QUEUE QUICK REFERENCE

STRUCTURE:

array of size N
front (first), rear (next free)
initially front = rear = 0

OPERATIONS (waste slot convention):

enqueue(x):
if $(\text{rear} + 1) \% N == \text{front}$: FULL
arr[rear] = x
rear = $(\text{rear} + 1) \% N$

dequeue():
if front == rear: EMPTY
x = arr[front]
front = $(\text{front} + 1) \% N$
return x

CONDITIONS:

empty : front == rear
full : $(\text{rear} + 1) \% N == \text{front}$
size : $(\text{rear} - \text{front} + N) \% N$

OPTIMISATION (power of two):

mask = capacity - 1
rear = $(\text{rear} + 1) \& \text{mask}$
front = $(\text{front} + 1) \& \text{mask}$

COMPLEXITY: $O(1)$ all operations.

Memory Aids & Mnemonics

"Modulo Wrap, No Shift"

The circular queue avoids all element shifting through modulo arithmetic.

"Front Catches Rear When Full"

$(\text{rear} + 1) \% N == \text{front}$ — rear is one step behind front when the queue is full.

"Equal Means Empty"

front == rear — when both pointers meet, the queue is empty.

Common Interview & Certification Questions

1

Q1: How does a circular queue differ from a linear queue?

A circular queue connects the last position to the first, reusing vacated slots via modulo arithmetic. A linear array queue cannot reuse space without $O(n)$ shifting, and becomes unusable once `rear` reaches the end.

2

Q2: How do you detect if a circular queue is full?

Common method (waste-slot): `(rear + 1) % capacity == front`. Alternative: maintain a separate `count` variable and check `count == N`.

3

Q3: Can we use all N slots without wasting one?

Yes — by maintaining a separate `count` variable or a boolean `full` flag. The count approach is simpler and less error-prone.

4

Q4: Why use power-of-two capacity?

To replace the modulo operation with a faster bitwise AND: `rear = (rear + 1) & mask`. This avoids the integer division instruction, saving 1–2 CPU cycles per operation.

5

Q5: What happens on enqueue when full (drop-tail policy)?

The operation fails and returns `false` or `null`. No data is overwritten. The caller is responsible for handling the rejection.

6

Q6: How do you resize a circular queue?

Allocate a new larger array. Copy elements in order starting from `front`, wrapping around as needed. Reset `front = 0`, `rear = size`. Complexity: $O(n)$.

7

Q7: Is a circular queue thread-safe by default?

No. Concurrent modifications require explicit synchronisation — either lock-based (mutex + condition variable) or lock-free (atomic operations with appropriate memory ordering).

Recommended Next Topics



Deque (Double-Ended Queue)

Extends the circular concept to both ends of the queue, enabling $O(1)$ insertion and deletion at front and rear. Foundational for sliding window algorithms and palindrome checking.



Ring Buffer in Operating Systems

Deep dive into `kfifo` in the Linux kernel — a real-world, production-grade circular buffer used for driver-to-userspace data transfer at scale.



Lock-Free Data Structures

Explore concurrent circular queues using C++ atomics and memory ordering. Essential for high-throughput, low-latency systems such as financial trading platforms.



Producer-Consumer Pattern

The canonical use case for circular buffers. Study blocking and non-blocking variants, backpressure strategies, and real-world implementations in Java and Python.



Developed by **Talenciglobal** — advancing technical education through structured, industry-aligned learning resources.